FIELD ENTITY · 개념 설명 · 2026-10-12

갱신 LOD — 안 보이는 몬스터는 덜 계산한다

몬스터를 **없애지 않고**, 화면 밖에서 아무도 보지 않는 동안 **계산을 덜 자주, 덜 정교하게** 하는 것. 존재 층(누구를 켤까 = ⑤ 거리)이 정한 몬스터는 그대로 두고, 그 안에서 **얼마나 공들여 돌릴지**만 바꾼다.

무엇을	어떻게	왜	지킬 것
화면 밖에 있고 싸우지 않는 몬스터	애니메이션 · 판단 · HP바 · 회피를 덜 자주, 덜 정교하게	켜진 몬스터의 약 87%가 화면 밖인데 지금은 전부 매 프레임 계산한다	플레이어가 눈치채면 실패. 싸우는 몬스터는 절대 깎지 않는다

1. LOD는 두 종류가 있다 — 우리가 하는 건 오른쪽

LOD = Level of Detail. "중요하지 않은 것에는 덜 공들인다"는 같은 생각을 **그리기**에 쓰느냐 **계산**에 쓰느냐의 차이.

렌더링 LOD vs 갱신 LOD

렌더링 LOD — 멀수록 단순한 모델로 그린다



줄이는 것 = 그리는 비용 (GPU)

갱신 LOD — 안 보이면 덜 자주 계산한다



모델은 그대로. 계산하는 횟수와 정밀도만 바꾼다

줄이는 것 = 계산 비용 (CPU)

렌더링 LOD의 숫자는 이해를 돕는 예시다.

왜 오른쪽인가

그리기는 엔진이 이미 화면 밖을 건너뛴다(절두체 컬링 — 카메라에 안 잡히는 렌더러는 그리지 않음). 그런데 **계산은 안 건너뛴다.** 화면 밖 몬스터도 애니메이션 뼈를 계산하고, 행동 트리를 돌고, HP바 위치를 구한다. 쿼터뷰 근거리 카메라라 몬스터가 "멀리서 작게" 보일 일도 거의 없어서, 이 게임에서 줄일 몫은 **계산 쪽**이다.

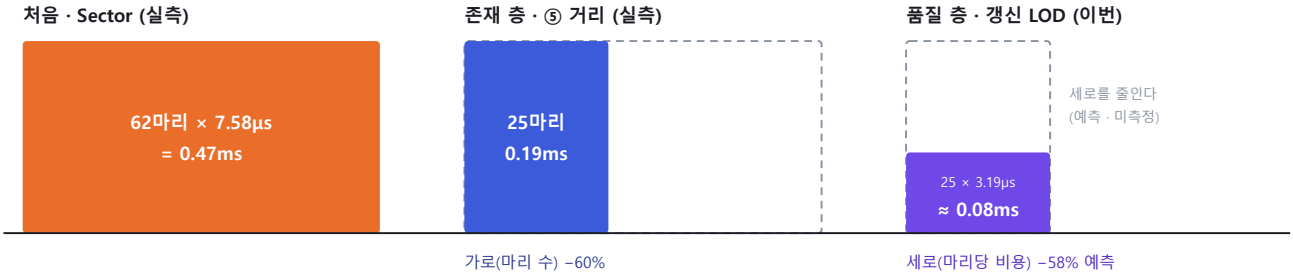
2. 비용 식에서의 자리 — 존재 층은 가로를, 갱신 LOD는 세로를 줄인다

실측 모델: 프레임 시간 = **1.307ms + 7.58µs × 살아 있는 몬스터** (정책 7개 평균점, r = 0.9998)

몬스터가 쓰는 시간 = 가로(마리 수) × 세로(한 마리당 비용)

프레임 = 1.307ms + (한 마리당 비용 × 살아 있는 몬스터 수)

가로 = 몬스터 수 · 세로 = 한 마리당 비용 · 넓이 = 몬스터가 쓰는 시간



고정 비용 1.307ms(맵 · 카메라 · UI 등)는 세 그림 모두 같아서 뺐다. 점선 = 이전 단계의 넓이.

솔직한 단서 — 지금 밀도에서는 이득이 작다

지금(⑤, 평균 25마리)은 몬스터 몫이 0.19ms뿐이다. LOD가 예측대로 돼도 **0.1ms 정도** 줄어서 측정 편차에 묻힌다. 그래서 증명은 **몬스터를 늘려서(밀도 ×1 → ×8)** 한다. 헤드라인은 "빨라졌다"가 아니라 **"같은 프레임 예산에 몬스터를 몇 배 돌릴 수 있나"**다(7절).

3. 몬스터 한 마리가 매 프레임 하는 일 — 실제 코드 기준

필드 몬스터 3종(Droid_Helmet_AR · RL · Tank 프리팹)과 `BaseMonsterController` · `NormalMonsterController` · `UI_MonsterHPBar`를 확인한 결과.

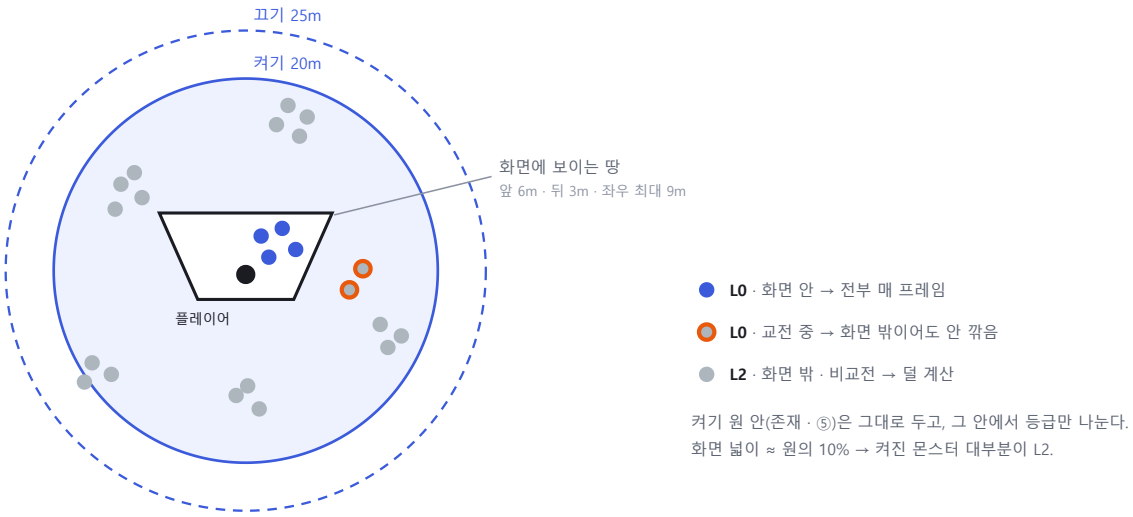
항목	어디서 · 지금 설정	하는 일	화면 밖이고 싸우지 않을 때 필요한가
Animator	엔진 · 프리팹의 `cullingMode = AlwaysAnimate` (0) — 3종 모두	매 프레임 뼈 수십 개의 자세 계산	**아무도 안 봄.** 단, 공격 판정이 애니메이션 이벤트라 시간은 흘러야 함
행동 트리	`BaseMonsterController.Update` → `_topNode.Evaluate()` 매 프레임	죽었나 · 귀환 · 감지 · 추적 · 공격 판단	필요하지만 **매 프레임일 필요는 없음**
HP바	`UI_MonsterHPBar.LateUpdate` · 마리마다 `WorldToScreenPoint`	머리 위 화면 좌표를 구해 바를 옮김	**아무도 안 봄** (바도 화면 밖)
회피	엔진 NavMeshAgent · `HighQualityObstacleAvoidance` (4) — 3종 모두	이웃 몬스터와 부딪히지 않게 속도 조정	이동은 필요, **정밀한 회피는 덜 중요**
고정 몫	엔진 · 트랜스폼 · 바운드 · 컬링 판정	존재하는 한 드는 비용	줄일 수 없음 — 이건 존재 층(가로)의 몫

항목별로 몇 μs인지는 **아직 재지 않았다**. 7.58μs를 항목별로 나눠 재는 것이 첫 단계다(7절 0단계).

4. 등급 나누기 — 원 안(존재)은 그대로, 원 안에서 등급만

화면에 보이는 땅은 **앞 6m · 뒤 3m · 좌우 최대 9m**(Game.unity 카메라 실측). 켜기 20m 원 넓이의 약 10%다.

위에서 본 필드 — 축척 1m = 5px



점 무리 = 스포너 하나의 몬스터들(실제로는 스포너당 최대 17마리). 주황 테두리 = 플레이어를 쫓는 중.

예측 문서(UpdateLOD_Predictions)와 다른 점

예측 문서에는 **L1(화면 안 · 12m 밖)** 등급이 있었다. 그런데 실제 화면은 앞 6m · 좌우 최대 9m라 **"화면 안인데 12m 밖"인 몬스터는 거의 없다.** 그래서 이 문서는 **L0 / L2 두 등급**으로 설명한다. 예측 문서도 실측 화면 크기로 고쳐야 한다.

5. 실제로 하는 조작 4가지 (A → D)

L2 몬스터에게만 적용한다. A는 엔진이 화면 판정을 직접 하고(렌더러 가시성), B · C · D는 우리가 매긴 등급으로 켠다.

A · Animator 컬링

프리팹 설정

`AlwaysAnimate` → `CullUpdateTransforms`. 화면 밖이면 뼈 자세를 계산해 쓰는 일을 건너뛰고, **상태 머신과 시간은 계속 흐른다.**

더 강한 `CullCompletely`(애니메이션 완전 정지)는 쓰지 않는다:

- 공격 판정이 애니메이션 이벤트(`OnAttackEvent` → `PerformAttackAction`)라 화면 밖 원거리 몬스터가 **쏘지 못한다**
- 화면 밖에서 스폰된 몬스터의 `Appear`가 멈춰 있다가 **화면에 들어올 때 재생된다** → 눈앞에서 등장하는 것처럼 보임

CullUpdateTransforms에서 이벤트가 실제로 발생하는지는 0단계에서 확인한다.

B · 행동 트리 분산

BaseMonsterController.Update

L2는 행동 트리를 **몇 프레임에 한 번**만 돈다. 이때 **몬스터마다 시작 프레임을 엇갈려** 한 프레임에 몰리지 않게 한다(아래 그림).

반응 지연: 4프레임에 한 번이면 144fps에서 약 28ms. 게다가 화면 밖이라 보이지 않는다.

C · HP바 정지

UI_MonsterHPBar.LateUpdate

화면 밖이면 `WorldToScreenPoint`와 바 이동을 건너뛴다.

지킬 것: 화면에 들어오는 **그 프레임에 바로 한 번 갱신**해야 바가 엉뚱한 곳에서 튀어나오지 않는다.

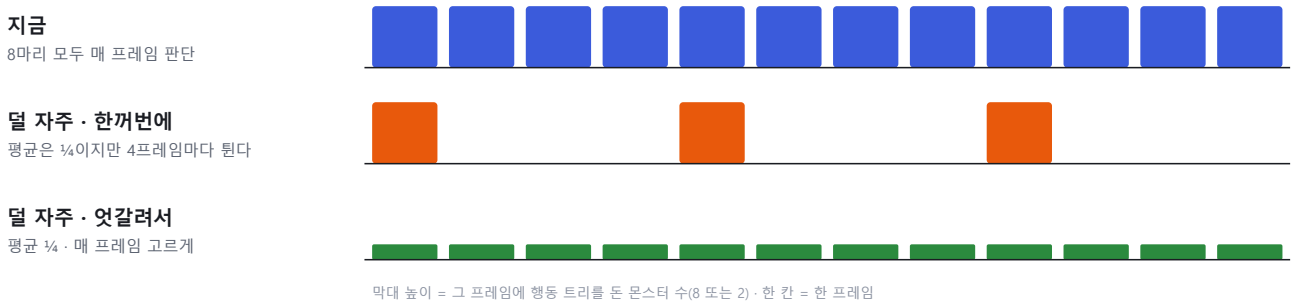
D · 회피 품질 낮추기

NavMeshAgent

`HighQuality` → 낮은 품질 또는 끔. **이동과 길찾기는 그대로**이고, 이웃과 비켜 가는 계산만 줄인다.

위험: 화면 밖에서 겹친 몬스터가 화면에 들어올 때 **겹쳐 보일 수 있다** → 측정 지표로 확인해야 한다.

B · 덜 자주 계산할 때 "엇갈림"이 필요한 이유 (몬스터 8마리 예시)



존재 층에서 "섹터 하나 = 119마리가 한 프레임에"가 끊김을 만들었던 것과 같은 문제다. 덜 자주 하더라도 한 프레임에 몰리면 튄다.

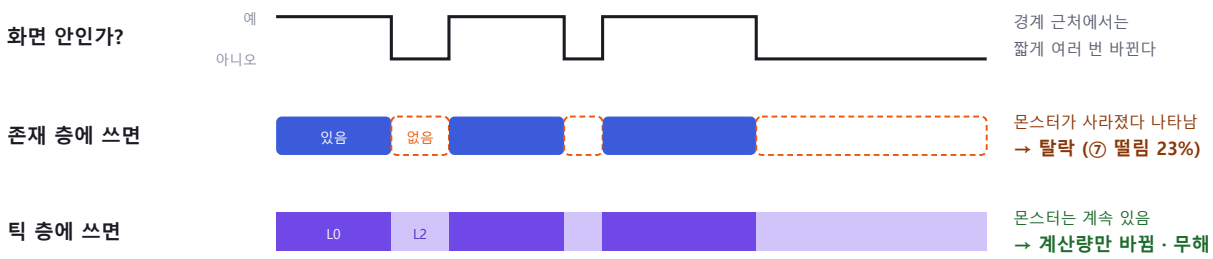
6. 눈치채지 못하게 — 지킬 규칙 3개

① 싸우는 몬스터는 깎지 않는다
교전 중이면 화면 밖이어도 L0. 존재 층의 "교전 유지" 규칙과 같은 논리 — **게임플레이에 닿는 것은 품질도 지킨다.**

② 화면에 들어오기 전에 L0
가시성은 렌더링 뒤에 갱신돼 **1프레임 늦다**(⑦ CullingGroup에서 확인). 판정 영역을 화면보다 조금 넓게 잡아, 늦게 반영돼도 보이지 않게 한다.

③ 등급이 떨려도 괜찮다
L0 ↔ L2가 오가도 몬스터는 **계속 존재**하고 계산량만 바뀐다. 존재 층에서는 치명적이던 떨림이 여기서는 무해하다(아래 그림).

같은 도구가 층에 따라 탈락도 되고 정답도 된다 — 가시성 판정(CullingGroup)



존재 층은 "있다/없다"라 떨림이 곧 깜빡임이다. 틱 층은 "얼마나 계산하나"라 떨림이 보이지 않는다.

7. 어떻게 증명하나 — 3단계

존재 층과 같은 방식: 측정 전에 예측을 박제하고, 같은 하네스 · 같은 개발 빌드에서 잰다.

0단계 → 1단계 → 2단계

0단계 · 분해

7.58µs를 항목별로 나눠 잰다
Animator · 행동 트리 · HP바 · 회피 · 고정 몫

→ 큰 항목부터 A~D 순서를 다시 정한다

1단계 · 계단

A → B → C → D 하나씩 켠다
켤 때마다 마리당 비용이 얼마나 내려가나

→ 어떤 조작이 얼마짜리인지

2단계 · 밀도 스윕

스포너당 마릿수 ×1 · ×2 · ×4 · ×8
× LOD 끔 / 켬 × 3회 = 24회
(무인 하네스 약 2시간)

→ 같은 예산에 몇 마리를 돌리나

0단계 없이 바로 구현하면 "Animator가 제일 비쌀 것"이라는 가정 위에서 순서를 정하게 된다. 가정부터 잰다.

판정 기준	통과 조건
비용 (헤드라인)	**같은 프레임 시간에 돌릴 수 있는 몬스터 수**가 몇 배인가 (예측: 약 2배 · 미측정)
교전 몬스터 무결성	화면 밖 교전 몬스터의 공격 누락 0 (공격 이벤트 수가 LOD 끔과 같음)
화면 진입 품질	화면에 들어오는 프레임의 자세 튐 · HP바 튐 · 겹침 0 — 새로 만들 지표
끊김	엇갈림 덕분에 6.94ms 초과 프레임 수가 LOD 끔보다 늘지 않음

8. 엔진들도 같은 개념을 기능으로 갖고 있다

엔진	기능	이 문서의 어디에 해당하나
Unity	`Animator.cullingMode`	A · 화면 밖 애니메이션 계산 생략
Unity	`CullingGroup` API	등급 판정(화면 안인가, 거리 밴드) — ⑦에서 이미 써 봄
Unity	`NavMeshAgent.obstacleAvoidanceType`	D · 회피 품질
Unreal	Significance Manager	"중요도"를 매겨 품질을 나누는 틀 — 등급 나누기
Unreal	URO(Update Rate Optimization) · Animation Budget Allocator	A + B · 애니메이션 갱신 빈도를 낮추고 예산 안에서 나눠 쓰기

한 줄로

존재 층이 **"몇 마리를 살려 둘까"**(가로)를 정했다면, 갱신 LOD는 **"살려 둔 몬스터를 얼마나 공들여 돌릴까"**(세로)를 정한다. 기준은 하나 — **플레이어가 보고 있거나 싸우고 있는가.** 아니면 덜 계산하고, 맞으면 하나도 깎지 않는다.

근거: 몬스터 프리팹 3종(Animator cullingMode 0 · 회피 4) · BaseMonsterController · NormalMonsterController(OnAttackEvent) · UI_MonsterHPBar · 비용 모델 = Policy_Result · 화면 = README · 예측 = UpdateLOD_Predictions(미측정)